ASSIGNMENT & ASSESSMENT

LAYANAN BEASISWA

Nama :

RAFSYANJANI EFFENDY (13020240299)

AHMAD ANDIRA ALAM (13020240339)

AMAR FARID A TOLOA (13020240287)

MUH. AIDIL (13020240190)

A.MUHAMMAD ADITYA IRAWAN (13020240081)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS MUSLIM INDONESIA

MAKASSAR

2026

DAFTAR ISI

BAB I PENDAHULUAN
U1 2
BAB II SKENARIO
U2 5
BAB III TAKTIK DAN POLA
U3–U4 5-9
BAB IV DESAIN
U5 10-12
BAB V EVALUASI DAN ADR
U6–U7 13-15
BAB VI PENUTUP 16
DAFTAR PUSTAKA 16
LAMPIRAN 17

BAB I

PENDAHULUAN

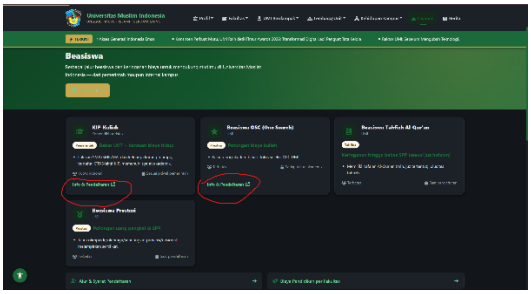
U1. Konteks dan ASR

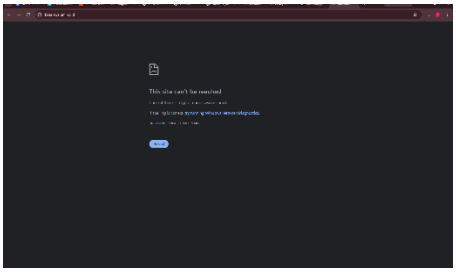
Ringkasan layanan

Subsistem layanan beasiswa adalah platform perangkat lunak terintegrasi untuk mengelola seluruh siklus administratif program beasiswa, yang mencakup modul pendaftaran, unggah berkas digital, seleksi/penilaian, dan pengumuman hasil.

Masalah utama beserta buktinya

Masalah utama layanan ini adalah potensi hilangnya dokumen pelamar dan tidak efisiennya proses administrasi. Hal ini dibuktikan dengan sering terjadinya kehilangan dokumen pelamar ketika pendaftaran masih menggunakan metode pengumpulan fisik atau manual. Dan ditandai dengan Website umi.ac.id/beasiswa dihalaman page tersebut terdapat beberapa pilihan kategori Beasiswa namun ketika Info dan Pendaftaran di klik website tidak dapat memuat page nya.





Stakeholder

Stakeholder	**Kelompok**	**Kebutuhan dan ekspektasi**	**Power / interest**	**Strategi**
Mahasiswa	Pengguna akhir	Waktu unggah dokumen < 10 detik, server 100% stabil 3 hari sebelum deadline, jaminan keamanan/enkripsi data pribadi	Rendah / tinggi	*Keep informed*
Tim penilai (*reviewer*)	Pengguna akhir	Akses berkas pelamar cepat, penilaian tersimpan aman, rekap otomatis	Tinggi / tinggi	*Manage closely*
Software engineer	Tim pengembang	Arsitektur modular, mudah diperbarui, dokumentasi jelas	Tinggi / tinggi	*Manage closely*
System administrator	Tim operasional	*Deployment* mudah, pemantauan server jelas, tidak membuat *server* kampus *down*	Tinggi / rendah	*Keep satisfied*

Direktur kemahasiswaan	Manajer bisnis	Seleksi tepat waktu, laporan pendaftar akurat, data pelamar aman	Tinggi / tinggi	*Manage closely*
Penyedia *cloud storage*	Pihak eksternal	Pemanggilan API sesuai standar, tidak melampaui batas *rate limit*	Rendah / rendah	*Monitor*

Kebutuhan Signifikan Secara Arsitektural (ASR)

No	Kategori ASR	Deskripsi Kebutuhan Signifikan (ASR)	**Alasan Signifikansi Arsitektural**
1	Performance / Kinerja	Sistem harus mampu menangani lonjakan akses tinggi secara bersamaan (high concurrency) saat hari-hari terakhir pendaftaran beasiswa dan pengumuman hasil seleksi.	Kebutuhan ini sangat krusial untuk menentukan kapasitas server, penerapan load balancing, serta mekanisme caching agar situs tidak mengalami down atau kelambatan akses saat diserbu ribuan pendaftar.
2	Security & Data Integrity / Keamanan	Sistem wajib mengamankan data pribadi sensitif pendaftar serta dokumen persyaratan yang diunggah (seperti KTP, transkrip nilai, slip gaji orang tua) dengan enkripsi ketat dan pembatasan akses (Role-Based Access Control).	Aspek arsitektur keamanan harus dirancang berlapis (baik enkripsi at rest maupun in transit) guna menjaga kerahasiaan data pribadi dan mencegah manipulasi dokumen atau hasil penilaian oleh pihak yang tidak berwenang.
3	Maintainability & Extensibility / Pemeliharaan & Skalabilitas	Modul alur seleksi dan penilaian berkas harus dirancang dengan struktur kode yang modular agar mudah disesuaikan jika ada perubahan kriteria atau bobot penilaian beasiswa di periode berikutnya.	Pemilihan pola arsitektur perangkat lunak yang modular (seperti pemisahan logika bisnis penilaian dari antarmuka pengguna) akan mempermudah pengembang dalam melakukan pemeliharaan dan pembaruan aturan seleksi tanpa harus merombak seluruh sistem.

Karakteristik pendorong

Kinerja (Performance Efficiency): Membutuhkan waktu respons sistem yang cepat dengan batas maksimal 2 detik saat menerima banyak akses dari pengguna.

Ketersediaan (Availability): Menargetkan tingkat ketersediaan layanan minimal sebesar 99,9% selama periode beasiswa berlangsung.

Keamanan (Security): Membutuhkan perlindungan 100% pada data pendaftar dan berkas sehingga data tersebut hanya dapat diakses oleh pihak yang berwenang.

BAB II
SKENARIO

BAB III
TAKTIK DAN POLA

U4. Pola Arsiterktur

A. Perbedaan Utama Gaya Arsitektur dan Pola Arsitektur

GAYA ARSITEKTUR	POLA ARSITEKTUR
• Cakupan: Seluruh Sistem (Macro View) • Menjawab: "Bagaimana seluruh Layanan Beasiswa disusun secara menyeluruh?" • Contoh: Microservices, Monolith	• Cakupan: Elemen / Komponen Spesifik •Menjawab: "Bagaimana alur/struktur kode & pemrosesan data ditangani?" •Contoh: Layered, Pipe-and-Filter

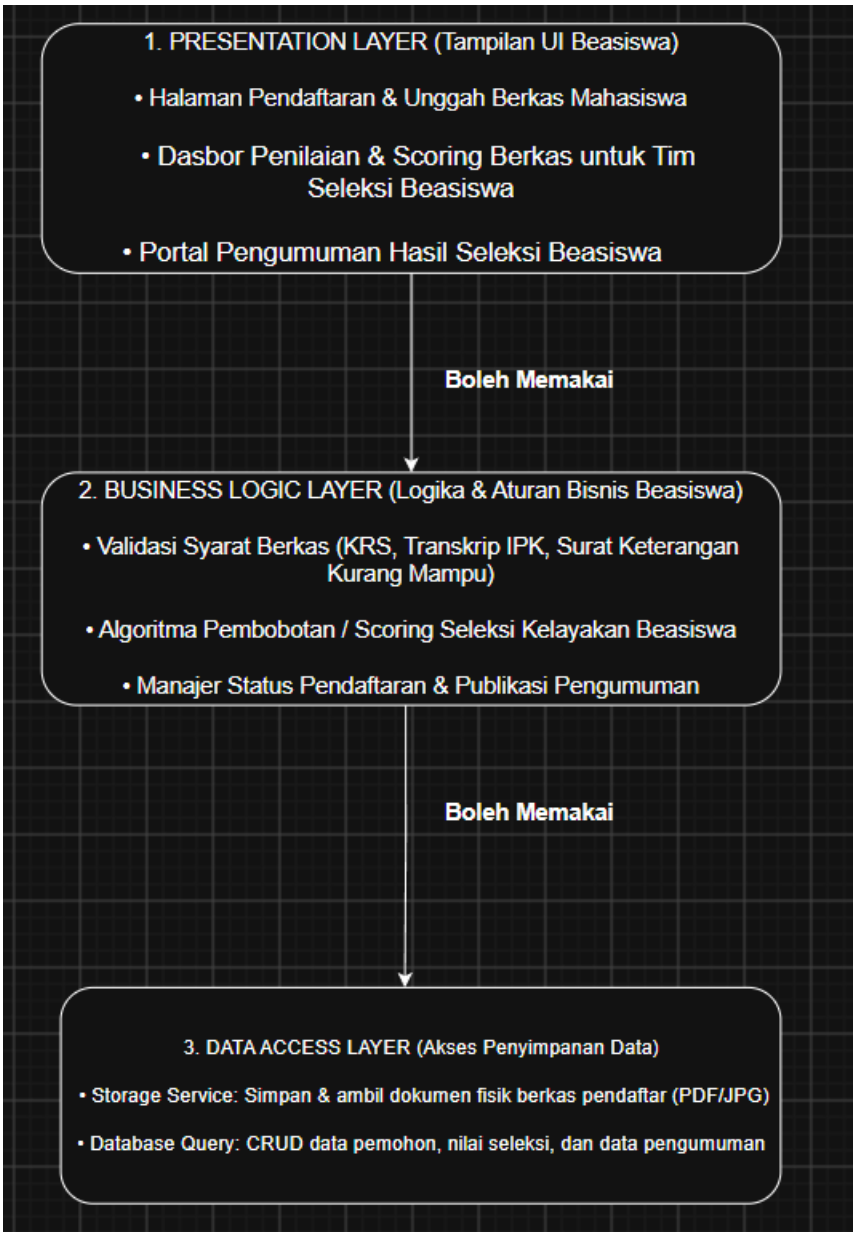
- **Gaya Arsitektur**: Mendefinisikan struktur tingkat tinggi dari seluruh Layanan Beasiswa (misalnya menggunakan gaya *Microservices* yang memisahkan modul pendaftaran, seleksi, dan pengumuman).

- **Pola Arsitektur**: Menjadi cetak biru spesifik untuk menyelesaikan masalah teknis di dalam subsistem Beasiswa (misalnya pola *Layered* untuk pengorganisasian kode internal, dan *Pipe-and-Filter* untuk pemrosesan seleksi berkas secara bertahap).

B. Pola Layered (Kategori Modul)

1. Diagram Pola Layered pada Layanan Beasiswa

Pola kategori modul ini membagi struktur kode Layanan Beasiswa menjadi 3 lapisan independen dengan aturan relasi *"allowed to use"* (lapisan atas hanya boleh memanggil lapisan langsung di bawahnya).



2. Alasan Pemilihan, Kelebihan, dan Kekurangan (Layered)

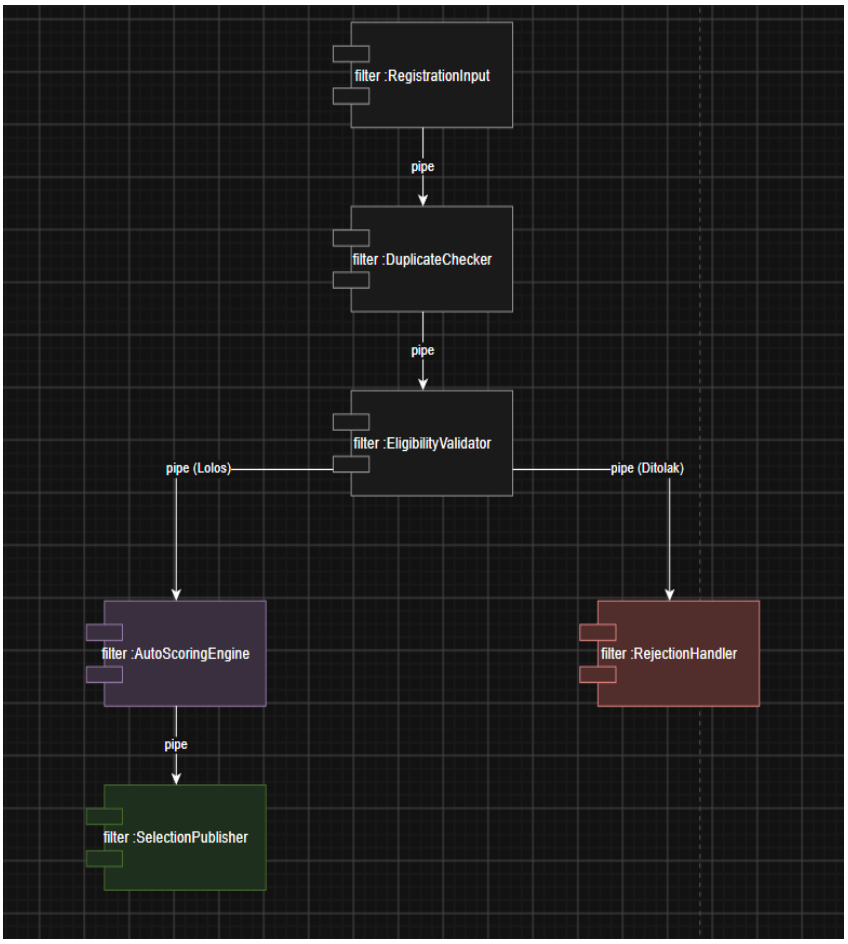
- **Alasan Pemilihan**: Memisahkan aturan kelayakan beasiswa (logika bisnis) dari tampilan antarmuka mahasiswa/dosen penilai dan mekanisme penyimpanan basis data. Jika aturan kuota atau bobot IPK berubah, perubahan hanya dilakukan pada *Business Logic Layer* tanpa merusak antarmuka UI.

- **Kelebihan (Modifiability & Testability)**:
    1. Kemudahan diubah (*Modifiable*): Modul basis data berkas atau tampilan pengumuman dapat diperbarui tanpa mengganggu modul lainnya.
    2. Kemudahan diuji (*Testable*): Aturan validasi skor beasiswa dapat diuji (*unit testing*) tanpa harus terhubung langsung ke basis data.
- **Kekurangan (Trade-off Performance):**
    1. Terdapat tambahan latensi (*overhead*) karena setiap *request* pendaftaran dan penilaian harus melintasi 3 lapisan berurutan.

C. Pola 2: Pola Pipe-and-Filter (Kategori C&C)

1. Diagram Pola Pipe-and-Filter pada Alur Seleksi & Berkas Beasiswa

Pola *Component-and-Connector* (C&C) ini digunakan untuk memproses data pendaftaran beasiswa yang masuk melalui serangkaian tahapan pemrosesan (*filter*) yang dihubungkan oleh saluran data (*pipe*).



2. Deskripsi Alur Komponen (*Filter*) dan Saluran (*Pipe*)

Berdasarkan diagram arsitektur sistem layanan beasiswa yang dirancang, alur kerjanya terbagi menjadi beberapa tahapan berikut:

- **:RegistrationInput (Filter 1 - Gerbang Pendaftaran):** Berfungsi sebagai titik masuk utama penerimaan data formulir pendaftaran dan berkas persyaratan dari mahasiswa. Data mentah ini dialirkan ke tahap berikutnya melalui pipe.
- **:DuplicateChecker (Filter 2 - Pemeriksa Duplikasi):** Berfungsi untuk menyaring dan memeriksa apakah mahasiswa yang bersangkutan sudah pernah mendaftar sebelumnya, guna mencegah terjadinya duplikasi data pendaftaran (*double entry*) di dalam sistem.
- **:EligibilityValidator (Filter 3 - Validasi Persyaratan Utama):** Berfungsi sebagai pusat penyaringan kelayakan utama (seperti pengecekan batas minimum IPK dan status keaktifan akademik). Pada titik ini, alur data bercabang menjadi dua jalur penanganan berbeda:
    1. **Jalur Lolos (pipe (Lolos)):** Meneruskan data pendaftar yang memenuhi seluruh persyaratan administratif dan akademik.
    2. **Jalur Ditolak (pipe (Ditolak)):** Memisahkan data pendaftar yang tidak memenuhi kriteria untuk dialirkan menuju modul penanganan khusus.
- **:AutoScoringEngine (Filter 4 - Mesin Penilaian Otomatis):** Menerima data dari jalur lolos untuk menghitung dan memberikan pembobotan skor secara otomatis berdasarkan kriteria prestasi dan kelayakan beasiswa.
- **:SelectionPublisher (Filter 5 - Publikasi Kelulusan):** Menerima hasil akhir skor, memformat daftar nama, dan menerbitkan pengumuman resmi penerima beasiswa.
- **:RejectionHandler (Filter 6 - Modul Penolakan):** Menangani data pendaftar dari jalur gagal untuk memformat pesan dan alasan penolakan agar sistem dapat mengirimkan notifikasi kepada pendaftar yang bersangkutan.

3. Alasan Pemilihan, Kelebihan, dan Kekurangan (Pipe-And-Filter)

- **Alasan Pemilihan (*Rationale*):** Pola *Pipe-and-Filter* dipilih karena alur proses seleksi beasiswa bersifat sekuensial, bertahap, dan melibatkan banyak transformasi data (mulai dari input data, validasi, penilaian, hingga publikasi). Pola ini membuat setiap tahapan seleksi terisolasi ke dalam komponen mandiri sehingga alur kerja sistem sangat terstruktur menyerupai lini perakitan data (*data-processing pipeline*).

- **Kelebihan (*Advantages*):**
  1. **Modularitas Tinggi**: Setiap filter berdiri sendiri sebagai modul independen sehingga perubahan pada satu filter tidak merusak filter lainnya.
  2. **Kemudahan Pemeliharaan (*Modifiability*):** Memudahkan pengembang untuk menyisipkan aturan seleksi atau mengubah metode penilaian baru tanpa merombak sistem utama.
  3. **Fleksibilitas Alur Kerja:** Memungkinkan pencabangan logika secara bersih antara pendaftar yang lolos ke jalur penilaian (:AutoScoringEngine) dan yang ditolak ke modul penolakan (:RejectionHandler).

- **Kekurangan (*Disadvantages*):**
  1. **Potensi Beban Kinerja (*Performance Overhead*)**: Setiap filter harus membaca dan menulis ulang data ke *pipe* berikutnya, yang dapat memperlambat pemrosesan jika struktur data terlalu besar.
  2. **Standarisasi Format Data:** Seluruh filter harus menyepakati format pertukaran data yang seragam; kesalahan format pada satu filter dapat menyebabkan kegagalan baca pada filter berikutnya.

BAB IV
DESAIN

1. Diagram konteks

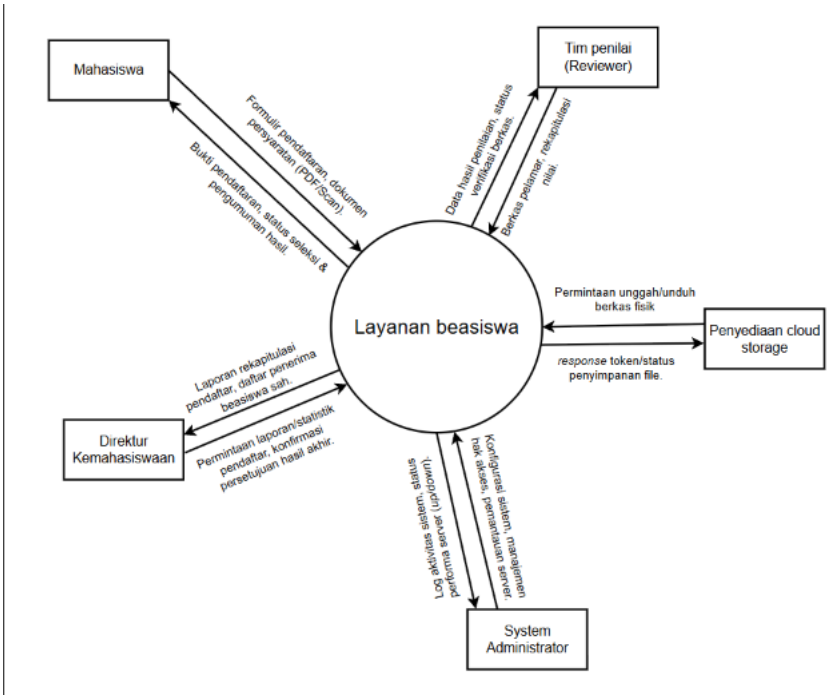


Diagram konteks ini menggambarkan batasan sistem layanan beasiswa sebagai proses pusat dengan 5 entitas eksternal yang berinteraksi:

1. Mahasiswa: Mengirimkan data formulir dan berkas persyaratan, serta menerima bukti pendaftaran dan informasi status seleksi.
2. Tim Penilai (Reviewer): Mengakses berkas pelamar untuk dinilai, lalu menginput hasil penilaian serta verifikasi berkas ke dalam sistem.
3. Direktur Kemahasiswaan: Meminta laporan statistik pendaftar dan memberikan persetujuan akhir terhadap penetapan penerima beasiswa.
4. System Administrator: Melakukan pemantauan performa server, mengelola konfigurasi hak akses, serta menerima log aktivitas sistem.
5. Penyedia Cloud Storage: Menerima permintaan *upload/download* berkas via API dan mengembalikan konfirmasi status penyimpanan dokumen.

2. Diagram arsitektur pola

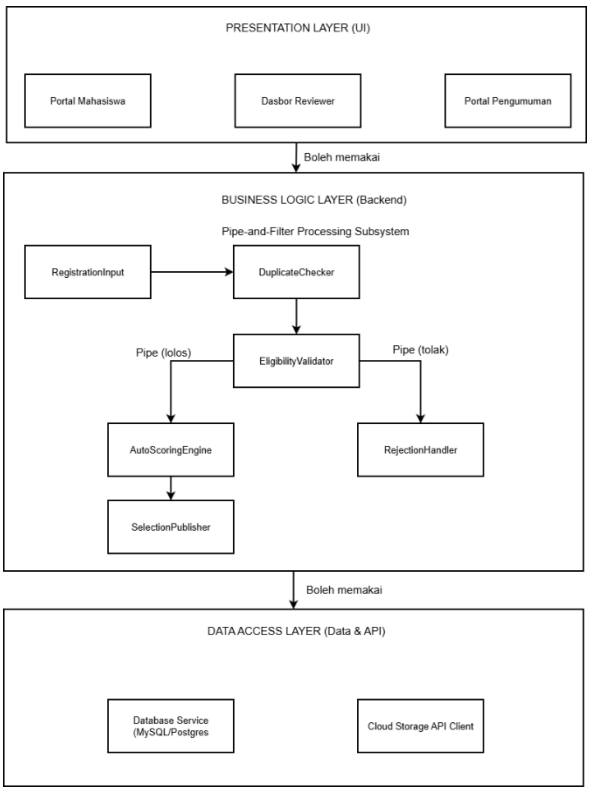


Diagram arsitektur ini menerapkan dua pola arsitektur utama dari U4:

1. Pola Layered Architecture (Kategori Modul): Digunakan sebagai struktur makro untuk memisahkan antarmuka pengguna (Presentation Layer), pemrosesan data & aturan bisnis (Business Logic Layer), serta akses penyimpanan data (Data Access Layer). Setiap lapisan hanya diperbolehkan mengakses lapisan satu tingkat di bawahnya (allowed to use) guna menjaga decoupling.
2. Pola Pipe-and-Filter (Kategori Component-and-Connector): Digunakan secara spesifik di dalam Business Logic Layer untuk menangani alur validasi dan seleksi berkas pendaftaran beasiswa. Berkas yang masuk dari RegistrationInput diproses berurutan melalui DuplicateChecker dan EligibilityValidator. Pendaftar yang lolos diteruskan ke AutoScoringEngine hingga publikasi kelayakan melalui SelectionPublisher, sedangkan pendaftar yang tidak memenuhi syarat dipisahkan ke RejectionHandler.

3. Diagram UML



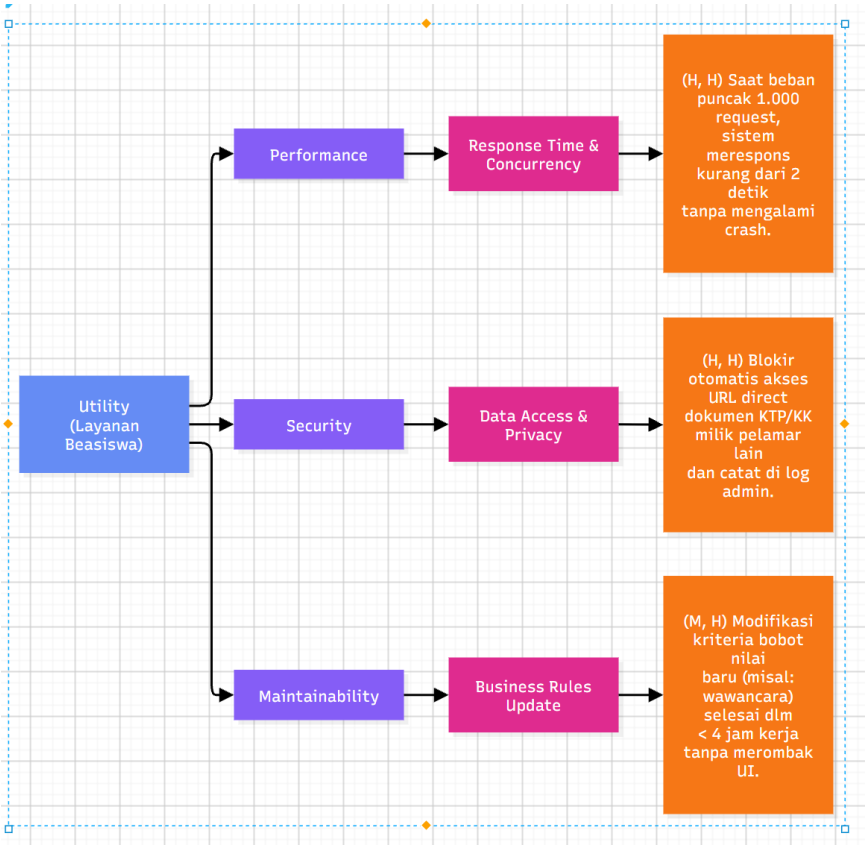
1. Mahasiswa (Primary Actor): Berperan sebagai pelamar beasiswa yang berinteraksi dengan sistem untuk mengunggah dokumen persyaratan secara mandiri, mengelola data profil pribadi, serta memantau status kelulusan seleksi secara real-time.
2. Tim Penilai / Reviewer (Primary Actor): Berperan mengevaluasi pendaftar dengan mengunduh/mengakses berkas pelamar, melakukan penilaian kelayakan berkas, dan melihat rekapitulasi penilaian otomatis.
3. Direktur Kemahasiswaan (Business Actor / Executive): Berperan memantau progres pelaksanaan seleksi beasiswa, melihat laporan statistik pendaftar yang akurat, serta memberikan persetujuan final (approval) penerima beasiswa.
4. System Administrator (Operational Actor): Berperan menjaga ketersediaan (availability) dan operasional sistem dengan memantau performa server, melakukan deployment pembaruan sistem, dan mengelola konfigurasi hak akses pengguna.
5. Software Engineer (Developer Actor): Berperan mengelola struktur modul sistem dan memperbarui dokumentasi teknis sistem.
6. Penyedia Cloud Storage (External Actor): Berperan sebagai penyedia repositori berkas digital berbasis API yang menerima dan menyimpan

dokumen persyaratan milik pelamar untuk menjamin keutuhan dan integritas data.

BAB V
EVALUASI DAN ADR

U6. Evaluasi

1. Pohon Utilitas Mini (Mini Utility Tree)



Pohon utilitas ini memetakan atribut kualitas ke skenario spesifik untuk memprioritaskan fokus arsitektur berdasarkan Nilai Bisnis (Business Value) dan Dampak Arsitektur (Architecture Impact), dinilai dengan skala Tinggi (H), Sedang (M), Rendah (L).

Akar: Kualitas Layanan Beasiswa UMI

Kinerja (Performance)

Skenario 1: Saat 3 hari sebelum penutupan (periode beban puncak), sistem menerima 1.000 *request* unggah berkas secara bersamaan. Sistem harus memproses dan merespons dalam waktu < 2 detik tanpa mengalami *crash* atau kegagalan muat halaman. (Prioritas: H, H)

Keamanan (Security)

Skenario 2: Entitas tidak berwenang mencoba mengakses tautan *URL direct* dokumen KTP dan KK milik pelamar lain. Sistem harus memblokir akses tersebut secara otomatis dan mencatat upaya pelanggaran ke dalam log aktivitas admin. (Prioritas: H, H)

Modifiabilitas (Maintainability)

Skenario 3: Direktur Kemahasiswaan memutuskan menambah kriteria bobot penilaian baru (misal: nilai wawancara) pada periode beasiswa tahun depan. Tim pengembang dapat memodifikasi logika *scoring* dalam waktu < 4 jam kerja tanpa harus merombak struktur UI/UX. (Prioritas: M, H)

3. Analisis ala ATAM

Risiko (Risk):

1. Ketergantungan API *Cloud Storage* dapat menurunkan Ketersediaan layanan jika terjadi gangguan jaringan.
2. Pola *Pipe-and-Filter* berisiko menambah latensi Kinerja akibat beban baca-tulis antar-*filter* saat data pendaftar membludak.

Bukan Risiko (Non-Risk): Aturan ketat pada *Layered Architecture* menjamin perubahan tampilan (UI) tidak akan merusak logika bisnis (*AutoScoring*).

Titik Sensitivitas (Sensitivity Point): Waktu respons sistem sangat bergantung pada kecepatan *bandwidth* jaringan menuju *Cloud Storage*.

Titik Trade-off (Trade-off Point): Penerapan enkripsi dan kontrol akses (RBAC) memaksimalkan Keamanan, namun harus mengorbankan sedikit Kinerja karena dekripsi memakan beban CPU.

4. Penjelasan Apakah Masalah Utama Teratasi Ya, masalah utama teratasi secara arsitektural:

1. **Risiko Dokumen Hilang:** Teratasi dengan *Cloud Storage* (Diagram U5). Berkas pendaftar tersimpan secara digital dengan integritas tinggi dan bebas dari risiko kerusakan/kehilangan fisik.
2. **Risiko Sistem Down/Error:** Teratasi lewat Pola *Layered* yang mendelegasikan beban *file* berat ke *Cloud*, sehingga server kampus fokus menangani tingginya lonjakan akses. Selain itu, alur data tertata rapi melalui *Pipe-and-Filter* guna mencegah *bottleneck*.

U7. Architecture Decision Records (ADR)

ADR 001: Pendelegasian Penyimpanan Berkas ke Cloud Storage

Status: Disetujui

Konteks: Dokumen fisik rentan hilang, dan server kampus sering *down* saat *deadline* karena tidak kuat menahan beban penyimpanan dan *bandwidth* dari ribuan berkas.

Keputusan: Memindahkan penyimpanan *file* ke *Cloud Storage API* (*Data Access Layer*). Database lokal hanya menyimpan URL dokumen.

Konsekuensi:

- *(+)* Beban server berkurang drastis (layanan stabil) dan dokumen dijamin aman dari kehilangan.
- *(-)* Muncul ketergantungan pada *cloud* pihak ketiga (jika API gangguan, fitur unggah lumpuh).

ADR 002: Kombinasi Pola Layered dan Pipe-and-Filter

Status: Disetujui

Konteks: Aturan seleksi beasiswa itu sekuensial (bertahap) dan rutin berubah. Jika menggunakan monolitik, mengubah aturan berisiko merusak antarmuka (UI) atau basis data.

Keputusan: Memakai arsitektur *Layered* (memisahkan UI, Logika, Data). Khusus pada modul Logika, dipakai *Pipe-and-Filter* untuk alur validasi dan *scoring*.

Konsekuensi:

- *(+)* Modifiabilitas sangat tinggi. Filter aturan baru mudah disisipkan tanpa merombak sistem atau merusak UI (*Non-Risk*).
- *(-)* Ada sedikit tambahan latensi (*overhead*) karena data harus dibaca dan diproses melewati banyak lapisan dan *filter* secara berurutan.

BAB VI
PENUTUP

DAFTAR PUSTAKA

[1] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Addison-Wesley Professional, 2021.

[2] M. Richards and N. Ford, Fundamentals of Software Architecture. O'Reilly Media, 2020.

[3] M. Nygard, "Documenting Architecture Decisions," Cognitect Blog, Nov. 15, 2011.

[4] A. L. Mendelow, "Environmental scanning: the impact of the stakeholder concept," in Proc. 2nd International Conference on Information Systems, 1981.

[5] R. E. Freeman, Strategic Management: A Stakeholder Approach. Pitman, 1984.

LAMPIRAN

ambuk	Nama	Kode yang dikerjakan	Bukti kontribusi (tautan)	Keterangan (Selesai / Tunda / Gagal)	
Ct.	13020240000	(Contoh) Ani	A2, A3	Riwayat berkas laporan/A2-stakeholder.md (tautan); commit "[A3] tambah batasan" (tautan)	Selesai
1	13020240339	Ahmad Andira Alam	U1	https://github.com/LAM2552/apl-tugas1-kelompok06/blob/main/UTS/laporan/U1.md	Selesai
2			U2		
3			U3		
4			U4		

5	**13020240081**	**A.Muhammad Aditya Irawan**	**U5**	**https://github.com/LAM2552/apl-tugas1-kelompok06/blob/main/UTS/laporan/U5.md**	**Selesai**
6	**13020240339**	**Ahmad Andira Alam**	**U6**	**https://github.com/LAM2552/apl-tugas1-kelompok06/blob/main/UTS/laporan/U6.md**	**Selesai**
7	**13020240339**	**Ahmad Andira Alam**	**U7**	**https://github.com/LAM2552/apl-tugas1-kelompok06/blob/main/UTS/laporan/U7.md**	**Selesai**